`brain/carl_reservoir.py` — Full Line-by-Line Walkthrough

Length: 197 lines. **Role:** the actual "cognitive" neural network — a Liquid State Machine (LSM) reservoir. This is the one file in the repo doing real recurrent-network math, and it's the only thing in the whole system that learns from experience while running (via RLS, explained below), rather than being either fixed rules (endocrine) or supervised-trained offline (the vision model).

What a "reservoir" is, in plain English, before the code

Normal neural networks you train with backpropagation: you adjust *every* connection weight based on the error. A reservoir computer does something different and much cheaper: you build a **large, fixed, randomly-connected recurrent network** (the "reservoir") and never train it at all — the random connections just churn the input into a rich, high-dimensional, constantly-changing pattern of activity. The *only* thing you actually train is a simple linear readout layer that reads out useful combinations of that churning activity. The idea (going back to Maass 2002 and Jaeger 2001, both cited in the docstring) is that a sufficiently large random recurrent network already contains almost any temporal pattern you'd want to extract, as long as you can find the right linear combination — so you skip the expensive part (training a recurrent network) and only do the cheap part (training a linear readout).

Lines 1–24: Header docstring

```python
"""
carl_reservoir.py — Liquid State Machine (LSM) Reservoir Brain (LSM-211, LSM-
...
    x(t+dt) = (1 - dt/τ) x(t) + (dt/τ) tanh(W_res @ x(t) + W_in @ u(t) + η(t)
    y(t) = W_out @ x(t)

W_out is trained online via Recursive Least Squares (RLS) at ≤5 Hz.
W_res and W_in are fixed (reservoir computing paradigm — only the readout lea
...
References:
    - Maass, Natschläger, Markram (2002): Real-time computing without stable
    - Jaeger (2001): The "echo state" approach to recurrent neural network tr
    - Sussillo & Abbott (2009): FORCE learning (RLS for reservoir readout)
"""
```

These two equations are the entire mathematical model, stated up front. First equation: the reservoir's internal state `x` at the next timestep is a blend of "mostly what it was before" and "a new nonlinear reaction to current input," where `tau` (τ) controls the blend ratio. Second equation: the actual motor output `y` is just a **linear** transform of the reservoir's current state — no nonlinearity in the readout itself, all the complexity lives in `x`. The three references are real, correctly-cited papers — Maass et al. coined "liquid state machines," Jaeger independently developed the closely related "echo state network" concept, and Sussillo & Abbott's FORCE learning is specifically the technique this file's `rls_update()` implements for training reservoir readouts online, in real time, while the network is running — as opposed to needing a separate offline training phase.

Lines 29–33: Class docstring

```python
class LiquidStateReservoir:
    """
    500-neuron LSM reservoir with RLS-trained readout.
    No backpropagation, no gradient descent — just echo state dynamics and or
    """
```

Worth internalizing this line if you're going to explain the project: **there is no backpropagation anywhere in this file.** If someone assumes "neural network" automatically means gradient descent/backprop, this is the correction — it's a fundamentally different training paradigm.

`__init__` (lines 35–96) — building the network

```python
    def __init__(self, N: int = 500, M: int = 25, K: int = 3,
                 spectral_radius: float = 1.0, sparsity: float = 0.15,
                 tau: float = 0.1, sigma_in: float = 0.1, seed: int = 42):
```

Default arguments here (`M=25`) don't actually matter in practice — `carl_simulation.py` always overrides them explicitly (`N=500, M=37, K=3, ...`) when constructing this class, so these defaults would only apply if someone instantiated `LiquidStateReservoir()` with no arguments at all, which doesn't happen anywhere in the current codebase.

```python
```

```
        self.N = N
        self.M = M
        self.K = K
        self.tau = tau
        self.sigma_noise_base = 0.02  # base exploration noise σ₀

        rng = np.random.RandomState(seed)
```

Stores the dimensions, and creates a **dedicated random number generator** seeded with the given seed, rather than using NumPy's shared global random state (`np.random.randn()` directly). This matters: it means this reservoir's random connectivity is reproducible and *isolated* from any other code elsewhere in the program that might also be drawing random numbers (e.g., `respawn_food()` in `carl_simulation.py` uses `np.random.uniform` on the global generator) — the two won't interfere with or depend on each other's call order.

python

```
        # ── W_in: Input weight matrix (N × M), dense Gaussian ──────────
        self.W_in = rng.randn(N, M) * sigma_in
```

`W_in` is a 500×37 matrix (with the actual `M=37` passed in from `carl_simulation.py`) — every one of the 500 reservoir neurons gets a random, independent connection strength from every one of the 37 sensory inputs. `rng.randn(N, M)` draws from a standard normal distribution (mean 0, std 1), then scaling by `sigma_in=0.1` shrinks those weights down to a std of 0.1 — small values, so no single sensory input can single-handedly saturate a neuron's activation.

python

```
        # ── W_res: Recurrent weight matrix (N × N), sparse, spectral-scaled
        W_raw = rng.randn(N, N)
        mask = (rng.rand(N, N) < sparsity).astype(float)
        W_raw *= mask
```

`W_raw` starts as a full dense 500×500 random matrix. `mask` is built by drawing uniform random numbers in [0, 1) for every entry and checking if they're below `sparsity=0.15` — meaning roughly 15% of entries will be `True`(1.0), 85% `False`(0.0). Multiplying element- wise zeroes out ~85% of the connections. **This is what makes it a *sparse* recurrent network** — most neurons are not directly connected to most other neurons, only about 15% of the ~250,000 possible neuron-to-neuron connections actually exist. Sparse connectivity is both more biologically realistic (real neural tissue isn't fully connected either) and computationally useful (it tends to produce richer, more varied

dynamics than a dense random matrix, which tends toward more homogeneous behavior).

```python
        eigenvalues = np.linalg.eigvals(W_raw)
        current_radius = np.max(np.abs(eigenvalues))
        if current_radius > 0:
            self.W_res = W_raw * (spectral_radius / current_radius)
        else:
            self.W_res = W_raw
        actual_sr = np.max(np.abs(np.linalg.eigvals(self.W_res)))
```

This is the single most important piece of math in the whole reservoir. The **spectral radius** of a matrix is the largest absolute value among its eigenvalues — for a recurrent network, this number determines whether the network's internal dynamics **die out** (radius < 1, any activity fades to zero over time), **blow up** (radius > 1, activity grows without bound / chaos), or sit at the **"edge of chaos"** (radius ≈ 1.0, the network can sustain long, complex, information-rich patterns of activity without either vanishing or exploding). This "edge of chaos" regime is a well-established finding in reservoir computing — it's where a reservoir has maximum computational capacity, because it's neither too simple (forgets everything instantly) nor too unstable (turns into unpredictable noise). The code computes the raw matrix's actual spectral radius (`current_radius`), then rescales the *entire matrix* by the ratio needed to bring that radius to exactly the target (`spectral_radius=1.0`, passed in from `carl_simulation.py`). The `if current_radius > 0` guard handles the edge case where the sparsity mask happened to produce an all-zero matrix (astronomically unlikely at N=500, but a legitimate defensive check). `actual_sr` re-measures after scaling, purely to report an accurate number in the print statement below (it should equal `spectral_radius` almost exactly, modulo floating-point rounding).

```python
        # ── W_out: Readout weight matrix (K × N), initialized to zero ────
        self.W_out = np.zeros((K, N))
```

This is a 3×500 matrix — one row per motor output (left wheel, right wheel, neck), one column per reservoir neuron. Starting at **all zeros** is a deliberate, meaningful choice (the comment explains it directly): a zero `W_out` means the reservoir contributes literally nothing to motor output at first — `y = W_out @ x = 0` no matter what `x` is. CARL's very first moments of operation are driven entirely by the CPG baseline and reflexes; the "cognitive" contribution only emerges gradually as RLS updates fill in `W_out` from zero.

```python
```

```
        self.x = np.zeros(N)
```

The reservoir's actual internal state (500 numbers) — also starts at zero, meaning no residual activity carries over from a previous run unless `load()` restores it later.

python

```
        self.rls_alpha = 1.0   # initial P scale
        self.rls_lambda = 0.999  # forgetting factor
        self.P = np.eye(N) * self.rls_alpha
```

Sets up the RLS algorithm's own internal bookkeeping matrix, `P` — a 500×500 identity matrix scaled by `rls_alpha=1.0` (meaning it starts as a plain identity matrix). `P` is explained properly in `rls_update()` below; briefly, it's an "inverse correlation matrix" that tracks which directions in the 500-dimensional reservoir-state space have already been well-explored by training, so RLS can take large, confident update steps in unexplored directions and small, cautious steps in already-well-calibrated ones. `rls_lambda=0.999` is a "forgetting factor" — very close to 1.0, meaning RLS mostly remembers all its accumulated experience, with only a very slow, gentle fading of very old data (a `lambda` further from 1.0 would make it adapt faster to recent data but "forget" older learned patterns faster too).

python

```
        self.total_rls_updates = 0
        self.last_prediction_error = np.zeros(K)

        print(f"[LSM-211] Reservoir initialized: {N} neurons, "
              f"spectral radius = {actual_sr:.4f}, "
              f"sparsity = {np.count_nonzero(mask)}/{N*N} "
              f"({100*np.count_nonzero(mask)/(N*N):.1f}%), "
              f"tau = {tau}s")
```

Two more counters used elsewhere (`total_rls_updates` is read by `carl_simulation.py`'s console diagnostics and final shutdown message; `last_prediction_error` is read by the RLS update-scheduling logic in `carl_simulation.py`, checking `error_norm = np.linalg.norm(self.reservoir.last_prediction_error)` to decide if training should run more urgently). The print statement is a genuinely useful startup sanity check — you can visually confirm the reservoir actually built itself with roughly 15% sparsity and spectral radius ≈1.0 every time the simulation starts, rather than trusting it blindly.

`step()` (lines 98–130) — one tick of the reservoir's own internal clock

```python
    def step(self, u: np.ndarray, noise_scale: float = 0.02, dt: float = 0.03
        eta = np.random.randn(self.N) * self.sigma_noise_base * noise_scale
```

Generates 500 random noise values (**note: this uses the global `np.random`, not the dedicated `rng` from `__init__`** — a small inconsistency; the reservoir's *structure* is built with an isolated, reproducible RNG, but its *runtime* noise draws from the shared global generator, meaning two runs with the same seed will build an identical reservoir but still behave slightly differently tick-to-tick due to this noise unless the global seed is also fixed elsewhere). The noise magnitude is `sigma_noise_base(0.02) × noise_scale`, where `noise_scale` is passed in from `carl_simulation.py` as `endocrine.get_exploration_noise_scale()` — directly tying norepinephrine level to how much random "jitter" gets injected into the reservoir's dynamics every tick.

```python
        leak = dt / self.tau
        drive = np.tanh(self.W_res @ self.x + self.W_in @ u + eta)
        self.x = (1.0 - leak) * self.x + leak * drive
```

This is the header equation made literal. `leak = dt/tau` — with `dt=0.033` and `tau=0.1` (the actual values used in `carl_simulation.py`), `leak ≈ 0.33`, meaning each tick blends in about a third new "drive" and keeps about two-thirds of the previous state — this is what makes it a **leaky integrator**: it changes gradually rather than snapping instantly to whatever the raw input suggests, giving the reservoir memory/inertia across ticks. `self.W_res @ self.x` is the recurrent contribution — how the network's *own current state* feeds back into itself through the sparse random connections. `self.W_in @ u` is the fresh sensory drive. Adding noise `eta` on top, then squashing the whole sum through `tanh` (which maps any real number into (−1, 1), providing the essential *nonlinearity* — without it, this would just be repeated linear algebra, which mathematically collapses into something far less expressive over time).

```python
        if np.isnan(self.x).any() or np.isinf(self.x).any():
            print("[WARNING][LSM-211] Reservoir state NaN/Inf detected. Soft-
            self.x = np.nan_to_num(self.x, nan=0.0, posinf=0.5, neginf=-0.5)
```

A defensive numerical-stability check specific to this class, in addition to the broader `S_unified`-wide check in `carl_simulation.py` — belt and suspenders. If `tanh`'s output

still somehow produced NaN/Inf (extremely unlikely given `tanh`'s bounded range, but possible if the *inputs* to it were already NaN/Inf from some upstream bug), replace bad values — notably clamping infinities to ±0.5 rather than ±1.0, a slightly more conservative "soft" reset that doesn't push the state to the extreme edge of `tanh`'s range.

```python
        y = self.W_out @ self.x
        y = np.clip(y, -2.0, 2.0)
        return y
```

The actual readout: `W_out` (3×500) times the current state (500,) gives 3 numbers — this is `y_cognitive` as seen in `carl_simulation.py`. Clamped to ±2.0 before returning as one more safety bound on top of whatever `write_motors()` does downstream later.

`rls_update()` (lines 132–169) — the online learning algorithm

This is the file's most mathematically involved method — genuine Recursive Least Squares, not a simplified approximation.

```python
    def rls_update(self, y_target: np.ndarray):
        phi = self.x  # current reservoir state
        y_pred = self.W_out @ phi
        error = y_target - y_pred
        self.last_prediction_error = error.copy()
```

`phi` (a common symbol in RLS literature for the "feature vector," here just the current reservoir state) gets the current output prediction subtracted from the target — the classic **prediction error**. Stored for later use by `carl_simulation.py`'s update-scheduling logic. `.copy()` matters here — without it, `last_prediction_error` would be a reference to the exact same underlying array as whatever `error` gets reused for, risking accidental aliasing bugs if `error` were mutated afterward (it isn't here, but it's good defensive practice).

```python
        P_phi = self.P @ phi
        denom = self.rls_lambda + phi @ P_phi
        if abs(denom) < 1e-12:
            return  # numerical protection
        gain = P_phi / denom
```

This is the **RLS gain vector** computation, straight from the standard RLS derivation: `k = P·ϕ / (λ + ϕᵀ·P·ϕ)`. Conceptually, `gain` answers "how much should each of the 500 neuron-weights move, per unit of prediction error, given what's already been learned about this particular direction in state-space." The `abs(denom) < 1e-12` check guards against division by a number so close to zero it would blow up numerically — an edge case that could theoretically arise if `phi` (the reservoir state) happened to be in a direction `P` has already "flattened" almost completely from repeated learning; returning early (skipping this update entirely) is the safe choice rather than risking a numerical explosion.

```python
        self.P = (self.P - np.outer(gain, phi @ self.P)) / self.rls_lambda
```

Updates the inverse correlation matrix itself using the standard **Sherman-Morrison-based RLS recursion** — this is what lets RLS avoid ever computing a full matrix inverse from scratch (which would be extremely expensive at 500×500, redone every learning step); instead it incrementally updates the *already-inverted* matrix using a rank-1 correction (`np.outer(gain, ...)` constructs exactly that rank-1 update). Dividing by `rls_lambda` (0.999) is the "forgetting" mechanism — it very slightly inflates `P` each update, which over time lets the algorithm continue adapting to new patterns rather than becoming permanently over-confident and rigid after enough updates.

```python
        p_norm = np.linalg.norm(self.P)
        if p_norm > 1e6:
            self.P *= (1e6 / p_norm)
```

A practical numerical-stability safeguard not typically part of the textbook RLS derivation: if `P`'s overall magnitude ever grows unreasonably large (a known failure mode of RLS over very long runs, sometimes called "covariance windup"), forcibly rescale it back down to a ceiling of `1e6`. This is a pragmatic engineering addition on top of the pure math, and a sensible one for a system meant to run continuously for a long time rather than a fixed, short training session.

```python
        for k_idx in range(self.K):
            self.W_out[k_idx] += error[k_idx] * gain

        self.total_rls_updates += 1
```

Lines 171–173: `get_state_norm()`

```python
    def get_state_norm(self) -> float:
        return float(np.linalg.norm(self.x))
```

Just the overall magnitude (L2 norm) of the 500-neuron state vector — a rough one-number summary of "how active is the reservoir right now," logged every tick in telemetry (`reservoir_norm` column).

`save()`/`load()` (lines 175–196)

```python
    def save(self, path: str):
        np.savez(path,
                 x=self.x, W_out=self.W_out, P=self.P,
                 W_in=self.W_in, W_res=self.W_res,
                 total_rls_updates=self.total_rls_updates)
        print(f"[GIS-901] Reservoir saved to {path}")
```

`np.savez` writes multiple named arrays into a single compressed `.npz` file — saves absolutely everything needed to fully resume: the current state, the *learned* readout weights, the RLS bookkeeping matrix, and — notably — even the fixed `W_in`/`W_res` matrices, even though those never change during training. Saving them anyway means a loaded reservoir is byte-for-byte reproducible regardless of what `seed` value a future run happens to pass into `__init__` — the loaded weights fully override whatever the constructor randomly generated.

```python
```

```
    def load(self, path: str):
        if os.path.exists(path):
            data = np.load(path)
            self.x = data['x']
            self.W_out = data['W_out']
            self.P = data['P']
            self.W_in = data['W_in']
            self.W_res = data['W_res']
            self.total_rls_updates = int(data['total_rls_updates'])
            print(f"[GIS-901] Reservoir loaded from {path} "
                  f"({self.total_rls_updates} prior RLS updates)")
            return True
        return False
```

This is the exact method whose `os.path.exists` guard was independently verified earlier in this review process — confirmed to gracefully return `False` (rather than crash) on a fresh clone where no saved checkpoint exists yet, letting `carl_simulation.py` continue with the freshly-initialized random reservoir from `__init__` instead.

Summary

This file is dense but internally consistent: build a big random sparse recurrent network tuned to the edge of chaos, run it forward with leaky-integrator dynamics, and train only a thin linear readout layer on top using real, correctly-implemented Recursive Least Squares — including the practical numerical safeguards (denominator check, P-norm clamp) that a from-scratch academic implementation might not bother with but a system meant to run continuously for hours genuinely needs. If someone asks you to explain "how does CARL learn," this file — not the endocrine system, not the grid cells — is the actual answer: it's the only component that adjusts its own weights based on experience while running.